

Getting Started with rladybugdb

Contents

Introduction	1
Installation	1
Opening a Database	1
Defining a Schema	2
Inserting Data	2
Querying Data	3
Graph Analysis	5
Cleanup	5
Type Mapping Reference	5
Real-world example: OpenFlights	6

Introduction

rladybugdb provides an R interface to **LadybugDB**, an embedded columnar graph database that uses the **openCypher** query language. LadybugDB is a fork of KuzuDB and supports the same Cypher dialect.

Starting with v0.2.0 the package uses **native Rcpp C++ bindings** compiled against the LadybugDB C library (liblbug). The C library is bundled with the package — no Python, no **reticulate**, and no extra installation step are required.

R → Rcpp (rladybugdb.so) → C API (lbug.h) → LadybugDB engine

Installation

Install from GitHub:

```
# install.packages("remotes")
remotes::install_github("rladybugdb/rladybugdb")
```

The **configure** script downloads the prebuilt liblbug binary for your platform at install time. After that the package is fully self-contained.

Verify the library is working:

```
library(rladybugdb)
ladybugdb_version() # e.g. "0.15.2"
#> [1] "0.15.2"
ladybugdb_is_installed() # TRUE
#> [1] TRUE
```

Note: ladybugdb_install() is deprecated and no longer does anything. You do not need to call it.

Opening a Database

lb_database() opens or creates a LadybugDB database. Pass **":memory:"** for a transient in-memory database (ideal for testing and interactive exploration):

```
library(rladybugdb)

db <- lb_database(":memory:")
conn <- lb_connection(db)
db
#> <lb_database> path=:memory: read_only=FALSE
conn
#> <lb_connection> db=:memory:
```

For a persistent on-disk database pass a directory path:

```
db <- lb_database("~/my_graph_db")
```

Use `lb_close()` to release resources when done:

```
lb_close(conn)
lb_close(db)
```

Defining a Schema

LadybugDB uses a **property graph** model: nodes belong to *node tables* and edges belong to *relationship tables*. Both are defined with Cypher DDL:

```
# Node tables
lb_execute(conn, "CREATE NODE TABLE Person (
  name  STRING,
  age   INT64,
  PRIMARY KEY (name)
)")
#> <lb_result> [1 x 1]
#>                                     result
#> 1 Table Person has been created.

lb_execute(conn, "CREATE NODE TABLE City (
  name     STRING,
  country  STRING,
  PRIMARY KEY (name)
)")
#> <lb_result> [1 x 1]
#>                                     result
#> 1 Table City has been created.

# Relationship table (directed: Person → City)
lb_execute(conn, "CREATE REL TABLE LivesIn (FROM Person TO City, since INT64)")
#> <lb_result> [1 x 1]
#>                                     result
#> 1 Table LivesIn has been created.
```

Inserting Data

Row-by-row with Cypher CREATE

```
lb_execute(conn, "CREATE (:Person {name: 'Alice', age: 30})")
#> <lb_result> (no columns)
lb_execute(conn, "CREATE (:Person {name: 'Bob', age: 25})")
```

```

#> <lb_result> (no columns)
lb_execute(conn, "CREATE (:City {name: 'London', country: 'UK'})")
#> <lb_result> (no columns)
lb_execute(conn, "CREATE (:City {name: 'Paris', country: 'France'})")
#> <lb_result> (no columns)

lb_execute(conn, "
  MATCH (p:Person {name: 'Alice'}), (c:City {name: 'London'})
  CREATE (p)-[:LivesIn {since: 2018}]->(c)
")
#> <lb_result> (no columns)
lb_execute(conn, "
  MATCH (p:Person {name: 'Bob'}), (c:City {name: 'Paris'})
  CREATE (p)-[:LivesIn {since: 2021}]->(c)
")
#> <lb_result> (no columns)

```

Bulk loading from an R data frame

For larger datasets `lb_copy_from_df()` writes the data frame to a temporary CSV and uses LadybugDB's COPY ... FROM loader — much faster than individual CREATE statements:

```

more_people <- data.frame(
  name = c("Carol", "Dave"),
  age  = c(35L,    28L)
)
lb_copy_from_df(conn, more_people, "Person")
#> <lb_result> [1 x 1]
#>
#> result
#> 1 2 tuples have been copied to the Person table.

lb_query(conn, "MATCH (p:Person) RETURN p.name AS name, p.age AS age ORDER BY p.name")
#>   name age
#> 1 Alice  30
#> 2  Bob  25
#> 3 Carol  35
#> 4  Dave  28

```

Loading from a CSV file

```
lb_copy_from_csv(conn, "/data/people.csv", "Person")
```

Querying Data

`lb_query()` — query directly to a data frame

```

df <- lb_query(conn,
  "MATCH (p:Person)-[:LivesIn]->(c:City)
  RETURN p.name AS person, c.name AS city, c.country AS country
  ORDER BY p.name")
df
#>   person  city country
#> 1 Alice London      UK

```

```
#> 2    Bob Paris France
```

lb_execute() — query to an lb_result

lb_result holds a lazy cursor. Convert it explicitly:

```
result <- lb_execute(conn,
  "MATCH (p:Person) RETURN p.name AS name, p.age AS age ORDER BY p.age")
result      # prints column schema
#> <lb_result> [4 x 2]
#>   name age
#> 1  Bob  25
#> 2  Dave 28
#> 3 Alice 30
#> 4 Carol 35
as.data.frame(result)
#>   name age
#> 1 <NA> NA
#> 2 <NA> NA
#> 3 <NA> NA
#> 4 <NA> NA
```

Parameterised queries

Use named parameters (prefixed with \$) instead of string interpolation. This avoids Cypher-injection bugs and handles type coercion automatically:

```
lb_execute(conn,
  "MATCH (p:Person {name: $name}) RETURN p.age AS age",
  parameters = list(name = "Alice"))
#> <lb_result> [1 x 1]
#>   age
#> 1  30
```

Supported parameter types: character, integer, double, logical.

As a tibble

```
result <- lb_execute(conn,
  "MATCH (p:Person) RETURN p.name AS name, p.age AS age ORDER BY p.age")
tibble::as_tibble(result)
#> # A tibble: 4 x 2
#>   name    age
#>   <chr> <dbl>
#> 1 Bob     25
#> 2 Dave    28
#> 3 Alice   30
#> 4 Carol   35
```

As an Arrow Table

```
result <- lb_execute(conn,
  "MATCH (p:Person) RETURN p.name AS name, p.age AS age")
as_arrow_table(result)
```

```
#> Table
#> 4 rows x 2 columns
#> $name <string>
#> $age <double>
#>
#> See $metadata for additional Schema metadata
```

Graph Analysis

When a query returns full node and relationship columns (i.e. RETURN p, r, c rather than individual properties), `as_igraph()` and `as_tbl_graph()` convert the result into graph objects.

```
result <- lb_execute(conn,
  "MATCH (p:Person)-[r:LivesIn]->(c:City) RETURN p, r, c")
g <- as_igraph(result)
print(g)
#> IGRAPH 17055b6 DN-- 4 2 --
#> + attr: name (v/c), _LABEL (v/c), age (v/n), country (v/c), _LABEL
#> | (e/c), since (e/n)
#> + edges from 17055b6 (vertex names):
#> [1] Alice->London Bob ->Paris
```

```
result2 <- lb_execute(conn,
  "MATCH (p:Person)-[r:LivesIn]->(c:City) RETURN p, r, c")
tg <- as_tbl_graph(result2)
tg
#> # A tbl_graph: 4 nodes and 2 edges
#> #
#> # A rooted forest with 2 trees
#> #
#> # Node Data: 4 x 4 (active)
#>   name   `_LABEL`   age country
#>   <chr> <chr>      <dbl> <chr>
#> 1 Alice Person      30 <NA>
#> 2 Bob   Person      25 <NA>
#> 3 London City        NA UK
#> 4 Paris City         NA France
#> #
#> # Edge Data: 2 x 4
#>   from to `_LABEL` since
#>   <int> <int> <chr>    <dbl>
#> 1     1     3 LivesIn  2018
#> 2     2     4 LivesIn  2021
```

Cleanup

```
lb_close(conn)
lb_close(db)
```

Type Mapping Reference

LadybugDB type	R type	Notes
INT8 / INT16 / INT32	integer	Exact up to 2^{53} ; no truncation for graph IDs
INT64 / SERIAL	double	
FLOAT / DOUBLE	double	
BOOLEAN	logical	
STRING / UUID	character	Days since Unix epoch Microseconds $\div 1e6 \rightarrow$ seconds since epoch
DATE	Date	
TIMESTAMP	POSIXct	
INTERVAL / DECIMAL / BLOB	character	
NULL	NA	Serialised as string Typed NA matching the column type
LIST / ARRAY	list column	Each cell is an R list
MAP	list with \$keys / \$values	
STRUCT	named list	
NODE	named list (_ID, _LABEL, properties)	
REL	named list (_SRC, _DST, _LABEL, _ID, properties)	Use <code>as_igraph()</code> to unwrap

Real-world example: OpenFlights

`example_openflights.R` (in the package root) shows a complete workflow with real data:

1. **Download** OpenFlights airport and route CSVs (~6 000 airports, ~66 000 routes)
2. **Load** into LadybugDB with `lb_copy_from_df()`
3. **Query** top hubs, country rankings, and a hub subgraph using Cypher
4. **Visualise** with `ggplot2`, `ggraph`, and the `maps` package

```
# From the package source directory:
Rscript example_openflights.R
```

Plots are written to `example_openflights_plots/`.